

C/C++言語入門

第 0.6 版

Copyright © 2026, Katsunori Nakamura

中村勝則

2026 年 9 月 23 日

免責事項

本書の内容は参考資料であり、掲載したプログラムリストは全て試作品である。本書の使用に伴って発生した不利益、損害の一切の責任を筆者ならびに IDEJ 出版は負わない。

目次

第1部 C言語の基礎	1
1 はじめに	2
1.1 C言語の歴史	2
1.1.1 C++	2
1.2 プログラミングに関する前提知識	2
1.2.1 機械語, アセンブリ言語, コンパイラ	2
1.2.2 インタプリタ, 仮想機械	3
1.3 本書で使用するC言語処理系	3
1.3.1 GNU Project について	3
1.3.2 Linux と GCC	4
1.3.3 本書で使用するツール	4
2 C言語プログラムの基本構造	5
2.1 main 関数の型, 戻り値	6
2.2 プリプロセッサ	6
2.3 プログラムのコンパイルと実行	6
2.4 関数定義の順序とプロトタイプ宣言	7
2.5 関数への引数の受け渡しについて	8
2.6 リテラル	8
2.6.1 数値リテラルのサフィックス	9
2.6.2 数値リテラルのプレフィックス	9
2.6.3 文字列リテラル	11
2.6.4 定数のマクロ定義	11
2.7 識別子の命名規則	12
3 変数, 型, 数値	13
3.1 変数の宣言	13
3.2 値の代入	13
3.2.1 定数としての変数	13
3.2.2 複合代入	14
3.3 基本的な型	14
3.4 変数のスコープと記憶期間	15
3.4.1 静的変数	16
3.4.2 グローバル変数	17
3.5 数値	18
3.5.1 整数が扱える値の範囲	18
3.5.1.1 最大幅整数型	20
3.5.2 浮動小数点数が扱える値の範囲	20
3.5.2.1 特殊な値	21
3.5.3 算術演算	23
3.5.3.1 インクリメント, デクリメント	23
3.5.4 浮動小数点数の誤差	24
3.5.5 キャストによる型の変換	25
3.5.6 数学関数	26
3.5.6.1 丸めモードの設定	26
3.5.6.2 乱数の生成	27

3.5.7	複素数型	29
3.5.7.1	虚数単位	29
3.5.7.2	実部と虚部の取得	29
3.5.7.3	複素数用の数学関数	30
3.5.7.4	複素数の出力	30
3.6	配列	31
3.6.1	配列宣言時の初期化	32
3.6.2	配列全体を複製する方法	33
3.7	文字と文字列	33
3.7.1	文字コードのリテラル表現	34
3.7.2	文字列の長さ	34
3.8	書式文字列と変換指定子	35
3.8.1	エスケープシーケンス	37
3.9	型の別名の定義	38
4	制御構造	39
4.1	条件分岐	39
4.1.1	if 文	39
4.1.1.1	条件判定式	40
4.1.1.2	_Bool 型と bool	41
4.1.1.3	浮動小数点数の判定	42
4.1.1.4	複素数の等値性の判定	44
4.1.2	条件演算子による条件式	44
4.1.3	switch 文	45
4.2	反復	47
4.2.1	while 文	47
4.2.1.1	do 文	48
4.2.2	for 文	48
4.2.2.1	コンマ演算子	49
4.3	無条件ジャンプ	50
5	構造体	51
5.1	構造体の様々な定義方法	52
5.1.1	構造体型の別名の定義	53
5.1.2	構造体をメンバとして持つ構造体	53
5.1.3	関数内で定義するローカルな構造体型	54
5.2	構造体の代入と複製	54
5.2.1	構造体のメンバとしての配列の扱い	55
5.2.2	関数呼び出しに伴う構造体の複製	55
6	メモリとポインタ	57
6.1	ポインタ	58
6.2	ポインタを介した値の受け渡し	59
6.2.1	関数間での変数の間接的な共有	61
6.3	配列のポインタとしての扱い	62
6.3.1	配列名とポインタ	63
6.3.2	ポインタの加算	63
6.3.3	配列とポインタの違い	64
6.3.3.1	関数定義の仮引数の書き方	64

6.4	ヌルポインタと NULL	65
7	入出力	67
7.1	ストリーム：入出力の対象	67
7.2	標準出力, 標準エラー出力	67
7.2.1	出力のリダイレクト	68
7.3	標準入力	69
7.3.1	入力のリダイレクト	70
7.3.2	入力用の様々な関数	70
7.3.2.1	文字列の読み込み	70
7.3.2.2	書式指定の読み込み：scanf	73
7.4	ファイル入出力	74
7.4.1	ファイルのオープンとクローズ	74
7.4.2	テキストファイルとバイナリファイル	74
7.4.3	ファイルへの出力	75
7.4.3.1	出力バッファの強制書き出し：fflush	76
7.4.4	ファイルからの入力	77
7.4.5	バイナリデータの扱い	79
7.4.6	既存のファイルの変更	81
7.4.7	ファイルのアクセス位置の操作	83
7.4.7.1	ファイルのアクセス位置の移動：fseek	83
7.4.7.2	アクセス位置の取得：ftell	84
7.4.7.3	ファイル先頭への移動：rewind	85
7.4.8	ファイルアクセスの際のエラーの確認：ferror	87
7.5	低水準入出力	88
7.5.1	低水準入出力の基本概念	88
7.5.2	ファイルのオープン：open	89
7.5.2.1	フラグとモード	89
7.5.3	書き込み：write	90
7.5.3.1	出力の確定処理	91
7.5.4	読み込み：read	91
7.5.5	ファイルのクローズ：close	92
7.5.6	入出力の効率化	93
8	分割コンパイルとプリプロセッサ	94
8.1	分割コンパイルのための基礎事項	94
8.1.1	extern による共有	96
8.1.2	static 宣言による共有範囲の制限	97
8.2	プリプロセッサ	98
8.2.1	ヘッダファイルの読み込み（#include）	98
8.2.2	定数マクロ（#define）	98
8.2.2.1	条件コンパイルへの応用	99
8.2.2.2	インクルードガード	100
8.2.2.3	処理環境に関する定数	101
9	実用的なプログラミングに必要な事柄	102
9.1	関数の再帰的定義	102
9.1.1	関数呼び出しにおけるメモリの管理の仕組み	103
9.1.2	関数の再帰呼び出しに関する問題点	104

9.1.2.1	スタックのサイズに関すること	104
9.1.2.2	計算規模に関すること	105
9.1.3	複雑な問題を再帰的関数で解決する例	107
9.2	関数定義の高度な方法	109
9.2.1	可変長の引数	109
9.2.1.1	受け取った可変長引数の個数について	111
9.2.1.2	受け取った可変長引数の型について	111
9.2.2	関数のポインタ	111
9.2.2.1	関数ポインタの配列	112
9.3	ビット演算	114
9.3.1	ビット演算の実用的な応用	115
9.3.2	ビットフィールド	117
9.3.2.1	C 言語の標準として使用できるビットフィールドの型	118
9.3.2.2	ビットフィールド使用上の注意	120
9.4	共用体	121
9.5	データのバイト順 (バイトオーダー) に関すること	124
9.5.1	バイト順の反転	124
9.5.2	ホスト/ネットワークバイト順変換関数	125
9.6	文字列操作関数	127
9.6.1	文字列の長さの取得: strlen	127
9.6.2	文字列の複製: strcpy	127
9.6.3	文字列の比較: strcmp	127
9.6.4	文字列の連結: strcat	127
9.6.5	文字の検索: strchr	127
9.6.6	文字列の検索: strstr	128
9.6.7	書式整形結果の文字列の作成: sprintf/snprintf	128
9.7	文字列解析関数	130
9.7.1	文字列からの値の読み取り: sscanf	130
9.7.2	文字列を整数値に変換する関数: strtol / strtoll / strtoul / strtoull	130
9.7.3	文字列を浮動小数点数に変換する関数: strtod / strtod / strtold	131
9.8	メモリの柔軟な取扱い	134
9.8.1	メモリの初期化	135
9.8.2	メモリの複製 (コピー)	136
9.8.3	メモリの比較	138
9.9	プログラム実行のためのメモリモデル	140
9.9.1	ヒープ領域の利用	141
9.10	列挙型: 名前付き整数の扱い	144
9.10.1	列挙子の値の割当てに関する注意点	144
9.10.2	列挙型を使用する目的に関すること	145
9.11	シグナル処理とジャンプ	147
9.11.1	シグナル処理の仕組みと流れ	147
9.11.2	シグナル処理関数: signal	148
9.11.3	シグナル処理のための変数	148
9.11.4	ハンドラからの強制的な制御移動	150
9.11.5	シグナルを他のプログラムに通知する方法	152
9.11.6	使用頻度の高いシグナル	152
9.12	日付, 時刻の扱い	154
9.12.1	UNIX 時間	154

9.12.2	時間を取り扱うための基本的な機能	154
9.12.2.1	日付, 時刻の書式整形	155
9.12.2.2	日付, 時刻の文字列からの読取り	156
9.12.3	高精度の時間計測	158
9.13	プログラムの一時停止	159
9.13.1	POSIX 環境での方法	159
9.13.2	Windows 環境での方法	160
9.13.3	待ち時間を実現する際の推奨されない方法	161
9.14	コマンドライン引数	163
9.14.1	main 関数の引数	163
9.14.2	コマンドラインの記述に関すること	163
9.15	プログラムの終了処理	166
9.15.1	exit 関数によるプログラムの終了	166
9.15.2	abort 関数によるプログラムの異常終了	168
第 2 部	C++	170
10	C++とは	171
11	C++の基礎	173
11.1	main 関数の記述の形式	173
11.2	クラスとオブジェクトに関する予備知識	173
11.3	C++の基本的なデータ型	173
11.3.1	変数宣言時の初期化	174
11.3.2	C++での複素数の扱い	174
11.4	入出力	175
11.4.1	iostream	175
11.4.2	書式制御の方法 (C 言語の変換指定子との違い)	175
11.4.2.1	桁数, 精度の設定	175
11.4.2.2	左右のアラインメントと埋め文字	176
11.4.3	ファイルストリーム	177
11.4.3.1	出力ファイルストリーム (ofstream)	177
11.4.3.2	入力ファイルストリーム (ifstream)	178
11.4.3.3	入出力ファイルストリーム (fstream)	179
11.4.3.4	バイナリファイルの扱い	179
11.4.3.5	入出力位置の制御	180
11.4.3.6	ファイルの一括読み込み	183
11.4.4	ストリームに対する各種の関数	184
11.4.4.1	ストリーム状態の検査	184
11.4.4.2	ストリーム状態の操作	185
11.4.4.3	入力補助関数	186
11.4.4.4	バイナリ入出力に関わる関数	187
11.4.4.5	その他の機能	188
11.4.5	C 言語の stdio の利用	189
11.5	名前空間	190
11.5.1	名前空間の指定	190
11.5.2	主な名前空間	190
11.5.3	独自の名前空間の定義	191
11.6	C++における型の考え方	192

11.6.1	論理型 (bool 型)	192
11.6.1.1	bool 値の入出力時の形式	193
11.6.2	特殊な数値	195
11.6.3	バイトを表す型 (byte 型)	196
11.6.4	C++の標準文字列クラス	197
11.6.4.1	string オブジェクトとして扱えるデータサイズ	197
11.6.4.2	文字列へ出力するストリーム: ostream	198
11.6.4.3	文字列から入力するストリーム: istream	199
11.6.5	明示的なヌルポインタ: nullptr	201
11.6.6	型名としての構造体名	202
11.6.7	型変換	202
11.6.7.1	static_cast	202
11.6.7.2	const_cast	203
11.6.7.3	reinterpret_cast	203
11.6.7.4	型で意味を表す設計	204
11.7	参照	204
11.7.1	関数呼び出しにおける参照の応用	204
11.7.1.1	構造体を参照で渡すケース	206
11.7.1.2	参照を返す関数	207
11.7.2	一時オブジェクトと const 参照	209
11.7.3	参照とポインタの違い	210
11.8	関数のオーバーロード	211
11.8.1	オーバーロードの設計上の注意	212
11.8.1.1	引数が参照である場合	213
11.8.2	デフォルト引数	213
11.9	auto による型推論	215
11.10	範囲に基づく for 文	217
11.10.1	要素を参照として受け取る場合	218
11.11	コンパイル時定数: constexpr	219
11.12	値のカテゴリ	220
11.13	例外処理	221
11.13.1	例外として扱われる事象	221
11.13.2	例外のハンドリング	222
11.13.3	例外の送出	222
11.13.4	noexcept による例外伝播の抑止	224
11.13.5	回復不能な異常と終了処理	225
11.14	日付や時間に関する機能	228
11.14.1	C 言語由来の日付, 時刻の処理	228
11.14.2	日付, 時刻の文字列変換	229
11.14.3	時刻点と時間間隔	230
11.15	C 言語標準ライブラリの利用	233
11.15.1	C 言語由来の標準ライブラリ	233
11.15.2	C++用ヘッダの読み込み	233
11.15.3	std 名前空間からの利用	234
11.15.4	拡張子.hを持つヘッダとの違い	235
11.15.5	C++で利用する際の注意	235

12 クラスとオブジェクト	237
12.1 クラスの定義	237
12.2 オブジェクトの独立性	238
12.3 アクセス指定	239
12.3.1 public と private	239
12.4 コンストラクタ	239
12.4.1 コンストラクタのオーバーロード	241
12.4.2 初期化の際の丸括弧と波括弧	241
12.4.3 暗黙の型変換を禁止する explicit 宣言	243
12.4.4 コピーコンストラクタ	243
12.4.5 単純なバイトコピーが可能な型: trivially copyable	244
12.5 デストラクタ	245
12.6 this ポインタ	246
12.7 const メンバ関数と mutable 指定	247
12.8 演算子のオーバーロード	248
12.8.1 複合代入演算子のオーバーロード	249
12.8.2 異なる型の値との演算とその実装	250
12.8.2.1 friend 関数	251
12.8.3 関数のように呼び出せるオブジェクト: operator()	251
12.9 カプセル化	253
12.9.1 不変条件 (インバリアント) の維持	253
12.9.2 インターフェースと実装の分離	254
12.10 メンバの静的宣言	255
12.11 friend によるクラス間の特別なアクセス	257
13 継承とポリモーフィズム	259
13.1 継承とは何か	259
13.2 継承の基本構文	260
13.3 基底クラスと派生クラス	261
13.4 メンバの継承とアクセス制御	261
13.5 コンストラクタとデストラクタの呼び出し順序	263
13.6 関数の再定義と名前の隠蔽	266
13.7 仮想関数と動的束縛	267
13.8 ポリモーフィズム	268
13.8.1 オブジェクトスライシング	270
13.9 抽象クラスと純粋仮想関数	271
13.10 仮想デストラクタ	272
13.11 継承設計に関する注意	276
13.12 C++におけるオブジェクトとオブジェクト指向プログラミング	276
14 オブジェクトの寿命と所有権	278
14.1 オブジェクトの寿命	278
14.1.1 オブジェクトの生成と破棄	278
14.1.2 スコープと寿命	279
14.1.3 関数から戻る際のオブジェクトの破棄	279
14.1.4 一時オブジェクトの寿命	279
14.1.5 静的記憶期間を持つオブジェクト	280
14.1.6 動的に生成されたオブジェクトの寿命	280

14.1.7	寿命を終えたオブジェクトへのアクセス	280
14.2	オブジェクトの所有権	282
14.2.1	所有と非所有	282
14.2.1.1	ポインタと所有権	282
14.2.1.2	所有権が不明確なポインタ	283
14.2.1.3	参照による非所有の利用	283
14.2.1.4	関数の引数と所有権	284
14.2.2	単独所有と共有所有	284
14.2.2.1	単独所有	284
14.2.2.2	共有所有	285
14.2.2.3	単独所有と共有所有の使い分け	285
14.2.3	所有権の移動	285
14.2.3.1	通常のポインタによる所有権の移動	286
14.2.3.2	コピーと移動の違い	286
14.2.3.3	移動後のオブジェクト	286
14.2.3.4	所有権の移動と単独所有	287
14.3	RAII	287
14.3.1	RAII による資源管理	287
14.3.2	複数の終了経路と RAII	288
14.3.3	ファイルの管理と RAII	288
14.3.4	標準ライブラリと RAII	289
14.3.5	RAII を利用する際の基本方針	289
14.4	スマートポインタ	290
14.4.1	単独所有のための unique_ptr	290
14.4.1.1	unique_ptr による所有権の移動	291
14.4.1.2	unique_ptr を関数へ渡す方法	291
14.4.2	共有所有のための shared_ptr	293
14.4.2.1	shared_ptr による所有権の解放	293
14.4.3	非所有の参照を表す weak_ptr	295
14.4.4	共有所有における循環参照	297
14.4.5	スマートポインタを選ぶ際の基本方針	298
14.5	Rule of Three/Five/Zero	298
14.5.1	特殊メンバ関数	298
14.5.2	Rule of Three	299
14.5.3	コピーを禁止する方法	301
14.5.4	Rule of Five	301
14.5.5	Rule of Zero	305
14.5.6	Rule of Three/Five/Zero の使い分け	305
15	クラス設計の実践	306
15.1	基本的なクラスの構成	306
15.2	小さなデータを表すクラス (値オブジェクト)	308
15.3	リソース管理のためのクラス設計	309
15.4	抽象クラスによるインターフェース設計	311
15.5	ポリモーフィズムを前提とした設計	313
15.6	コンポジションによる設計	315
15.7	よくある設計上の問題	317
15.8	設計指針のまとめ	318

16 型を抽象化するための機能	322
16.1 関数テンプレート	322
16.1.1 テンプレートにおける転送参照	324
16.2 クラステンプレート	326
16.3 値を抽象化するテンプレート	329
16.3.1 型の別名を定義する方法	331
16.4 変数テンプレート	334
16.5 型安全な列挙型：enum class	335
17 ラムダ式	337
17.1 ラムダ式の基本	337
17.1.1 ラムダ式の即時呼び出し	338
17.2 外部の変数の利用：キャプチャ	339
17.3 ラムダ式の実体	340
18 並行処理の基礎	342
18.1 並行処理の実行単位：thread	342
18.2 スレッドの終了待ち：join	343
18.3 スレッドの切り離し：detach	345
18.4 スレッドの休止：sleep_for	346
18.5 共有データとデータ競合	347
18.5.1 データ競合が発生する例	348
18.5.2 mutex による排他制御	350
18.5.2.1 lock_guard による安全なロック管理	352
18.5.3 atomic による単純な共有値の操作	353
19 プログラムの構成と分割	357
19.1 クラスの宣言とメンバ関数の定義	357
19.1.1 class 文の外部におけるメンバ関数の定義	358
19.1.2 宣言と定義の一致	358
19.2 クラス定義と実装のファイル分割	360
19.2.1 クラス定義とメンバ関数定義の分離	360
19.2.2 ヘッダファイルを記述する際の C++固有の注意	362
19.3 名前空間に属する関数やメンバ関数の定義	363
19.4 ヘッダファイルに定義を記述する場合	365
19.4.1 inline 関数	365
19.4.2 クラス定義内で定義されたメンバ関数	366
19.4.3 inline 変数	367
19.4.4 定義の重複に関する注意	367
19.5 静的データメンバの定義	369
19.6 テンプレートの定義とファイル分割	372
19.6.1 テンプレートの定義を利用側から参照できる必要性	372
19.6.2 テンプレート定義の別ファイル化	373
19.6.3 明示的インスタンス化	374
19.7 C 言語のプログラムとの連携	377
19.7.1 C と C++のリンケージの違い	377
19.7.2 extern "C"	378
19.7.3 C と C++の双方から利用する宣言	378
19.7.4 C と C++のオブジェクトファイルのリンク	379

20 STLの基礎	384
20.1 要素数を変更できる配列：vector	384
20.1.1 生成時の初期化	385
20.2 イテレータ	386
20.2.1 要素を書き換えないイテレータ：const_iterator	388
20.3 要素数が固定された配列：array	390
20.4 双方向リスト：list	391
20.5 両端キューのコンテナ：deque	393
20.6 キーと値の対応を管理するコンテナ：map	394
20.6.1 順序を管理しない高速な連想コンテナ：unordered_map	396
20.7 重複のない値を管理するコンテナ：set	397
20.7.1 順序を管理しない高速な集合コンテナ：unordered_set	398
20.8 主なコンテナに対して使用できるメンバ関数	400
20.8.1 STL が提供するアルゴリズム	401
20.8.2 コンテナ間の変換処理	403
21 string による文字列処理	407
21.1 文字と文字列の検索	407
21.2 文字列の分割	409
21.3 文字の種類の判定と変換	410
21.4 日本語文字列と文字コードに関する注意	412
21.5 正規表現による文字列処理	414
21.5.1 正規表現の基本的な要素	416
21.5.2 正規表現に一致する箇所の全検索	417
22 数値データの処理	419
22.1 乱数	419
22.1.1 乱数エンジン	419
22.1.2 一様乱数	422
22.1.2.1 整数の一様乱数	422
22.1.2.2 浮動小数点数の一様乱数	423
22.1.3 正規分布に従う乱数	424
22.1.4 様々な乱数分布	425
22.2 数値列の集計：numeric	426
22.2.1 総和と平均	426
22.2.2 分散と標準偏差	427
22.2.3 内積と累積和	429
22.2.4 連番の生成	431
22.3 複素数の扱い：complex	433
22.3.1 実部、虚部	433
22.3.2 基本的な関数	434
22.3.3 複素数に対応する数学関数	435
22.4 多倍長精度演算	437
22.4.1 GMP	438
22.4.1.1 GMP の主な型	438
22.4.1.2 GMP の主な関数	439
22.4.1.3 多倍長浮動小数点数の精度の設定と初期化	442

22.4.1.4	mpf_t の数学的な演算のための関数	443
22.4.1.5	比較, 判定のための関数	444
22.4.1.6	mpf_t の値の近さの判定	446
22.4.1.7	書式整形と出力	447
22.4.1.8	サンプルプログラム (1): 多倍長整数	449
22.4.1.9	サンプルプログラム (2): 多倍長有理数	450
22.4.1.10	サンプルプログラム (3): 多倍長浮動小数点数	451
22.4.1.11	mpz_t 型の乱数	451
22.4.1.12	C++における GMP の利用	454
22.4.2	MPFR	463
22.4.2.1	MPFR の主な型	463
22.4.2.2	オブジェクトの管理のための関数	465
22.4.2.3	値の設定と取得のための関数	466
22.4.2.4	演算のための関数 (1): 基本的な演算	468
22.4.2.5	比較, 判定のための関数	469
22.4.2.6	mpfr_t の値の近さの判定	470
22.4.2.7	整数値への丸めに関する関数	473
22.4.2.8	演算のための関数 (2): 累乗・平方根など	474
22.4.2.9	演算のための関数 (3): 指数関数, 対数関数	476
22.4.2.10	演算のための関数 (4): 三角関数	477
22.4.2.11	演算のための関数 (5): 双曲線関数	479
22.4.2.12	演算のための関数 (6): 特殊関数, 定数	480
22.4.2.13	MPFR のキャッシュの解放	482
22.4.2.14	mpfr_t の特殊な値	483
22.4.2.15	書式整形や入出力のための関数	484
22.4.2.16	mpfr_t 型の乱数	486
22.4.2.17	10 進桁数による精度の指定	489
22.4.3	MPC	491
22.4.3.1	MPC の型とオブジェクトの管理	492
22.4.3.2	値の設定と取得	493
22.4.3.3	MPC の丸め方法	495
22.4.3.4	実部, 虚部, 絶対値, 偏角	495
22.4.3.5	基本的な演算	496
22.4.3.6	累乗, 指数関数, 対数関数	498
22.4.3.7	三角関数, 双曲線関数	499
22.4.3.8	書式整形と入出力	501
22.4.3.9	サンプルプログラム: 1 の n 乗根	503
第 4 部 付録		505
A 定義済みマクロ		506
A.1	言語モードと規格に関するマクロ	506
A.2	コンパイラとターゲット環境に関するマクロ	506
A.3	ホスト実装とフリースタANDING実装	507
B サンプルプログラム		511
B.1	整数を 2 進数表記の文字列に変換する機能	511
B.2	高精度の時間計測 (C 言語用)	514

C	コンパイラと開発環境	517
C.1	GCC	517
C.1.1	基本的なコマンド書式	517
C.1.2	主なコンパイラオプション	518
C.1.3	最適化水準	519
C.1.4	文字のエンコーディングの指定	519
C.1.5	実行時のスタックサイズの指定	520
C.2	Clang / LLVM	522
C.2.1	LLVM IR とコンパイラの構成	522
C.2.2	clang と clang++ の基本的な使用方法	523
C.2.3	主なコンパイラオプション	524
C.3	MSVC (Microsoft Visual C++)	526
C.3.1	必要となるツールの導入	526
C.3.2	MSVC を使用するためのコマンド環境の起動	526
C.3.3	cl コマンド	527
C.3.4	主なコンパイラオプション	528
C.3.5	文字のエンコーディングの指定	529
C.3.6	実行時のスタックサイズの指定	530
C.3.7	開発ツールのアップデート	531
C.4	MSYS2 による Windows 開発環境	532
C.4.1	MSYS2 のインストール	532
C.4.2	MSYS2 の起動	532
C.4.3	開発ツールのインストール	533
C.4.4	パッケージの管理	534
D	ソフトウェア構築の支援ツール	535
D.1	make	537
D.1.1	基本的な使用方法	537
D.1.2	ソースファイルの変更に対する make の働き	540
D.1.3	擬似ターゲット	541
D.2	CMake	542
D.2.1	基本的な使用方法	543
D.2.2	生成物の格納位置の設定	547
D.3	Ninja	548
D.3.1	CMake による build.ninja ファイルの生成	549
D.3.2	Ninja による構築	549
E	動的ライブラリの構築	551
E.1	動的ライブラリの基本的な仕組み	551
E.1.1	公開シンボルと利用側プログラムとの結合	552
E.1.2	起動時の読み込みと実行中の読み込み	552
E.2	公開インターフェースの設計	552
E.2.1	公開用ヘッダファイルと C ABI	553
E.2.2	公開指定のためのマクロ	553
E.3	例) 数値集計ライブラリの実装	554
E.3.1	C によるライブラリの実装	555
E.3.2	C++ によるライブラリの実装	556
E.3.3	ライブラリを利用するプログラム	557

E.4	環境別の動的ライブラリ構築	558
E.4.1	Linux で GCC を使用する場合	558
E.4.2	Linux で Clang を使用する場合	559
E.4.3	macOS で Clang を使用する場合	559
E.4.4	Windows の MSYS2 UCRT64 環境で GCC を使用する場合	560
E.4.5	Windows の MSYS2 CLANG64 環境で Clang を使用する場合	560
E.4.6	Windows で MSVC を使用する場合	561
E.5	公開インターフェース設計上の注意	562
E.5.1	ABI 互換性の維持	562
E.5.2	メモリの管理とオブジェクトの寿命	562
E.5.3	C++ 例外の扱い	563
F	gnuplot	564
F.1	データファイルの形式	565
F.1.1	データファイルの区切り文字の指定	565
F.2	描画に関する各種のコマンド	567
F.2.1	グラフの色の指定	568
F.2.2	線の太さの指定	569
F.2.3	線種の指定	569
F.3	描画結果の画像ファイルへの保存	570
F.4	ファイルに記述されたコマンドの実行	571
F.5	グラフに日本語を使用する際の注意事項	572
F.6	各種のグラフ	573
F.6.1	マーカープロット (点グラフ)	573
F.6.2	棒グラフ	574
F.6.3	散布図	576
F.6.4	ヒストグラム	578
F.6.5	3次元プロット (ワイヤーフレーム)	581
F.6.6	ヒートマップ	583
F.6.7	等高線	585